

Machine Learning Project 1

1. Data/Domain Understanding and Exploration (3 pages, 30%)

1.1. Meaning and Type of Features; Analysis of Distributions (3-4)

The data frame has 402005 rows and 12 columns. For the purposes of this report we will drop the public reference column as it has no impact on the price.

1. The **year_of_registration** column consists of numeric values and is stored as a

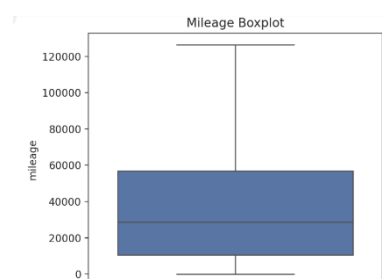
```
51] #Descriptive Stats.
adv[['mileage', 'year_of_registration', 'price']].describe().round().T
```

	count	mean	std	min	25%	50%	75%	max
mileage	401878.0	37744.0	34832.0	0.0	10481.0	28630.0	56876.0	999999.0
year_of_registration	368694.0	2015.0	8.0	999.0	2013.0	2016.0	2018.0	2020.0
price	402005.0	17342.0	46437.0	120.0	7495.0	12600.0	20000.0	999999.0

float data type to one decimal place. This column indicates the year in which the vehicles were registered and hence the age of the vehicle. The mean

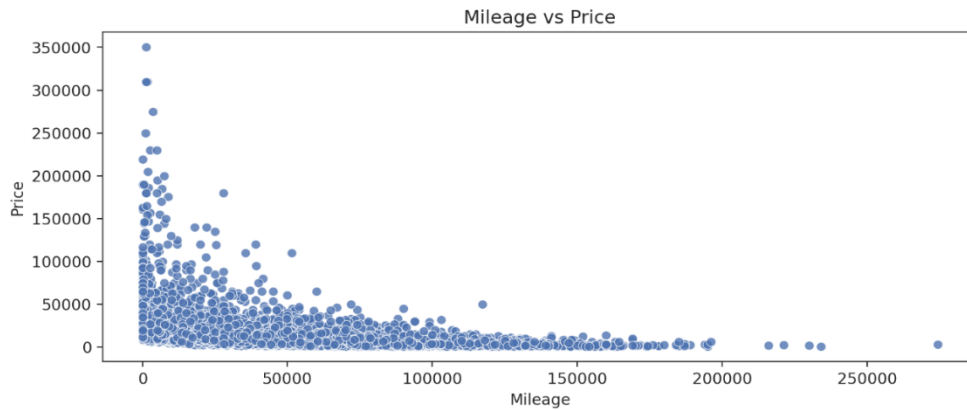
year is 2015 and the years range from 999 to 2020. 75% of the years are between 2013 and 2020 and just a small percentage is below the year 2000.

2. The **standard_make** column indicates the vehicle manufacturer and consists of 110 different makes. The values are recorded as object value data which indicates the presence of non-numerical values. This data may be a useful indicator of price although manufacturers may offer a variety of models at different price ranges
3. The values in the **reg_code** column represent information from the vehicle license plate. The values are stored as object data type. This means there may be non-numeric as well as the numeric values we can see at first glance. From these values we should be able to ascertain the year of registration and hence the age of the vehicle. In this respect it serves the same purpose as the **year_of_registration** column.
4. The **mileage** column has numerical values to one decimal place stored as float data type. The values are quantitative and represent how many miles the vehicle has travelled and is likely to influence the price of the vehicle. The values range from 0 to 999,999 and the mean mileage is just under 40,000 miles. From a boxplot with the outliers removed, we see the mileage distribution has positive skew.

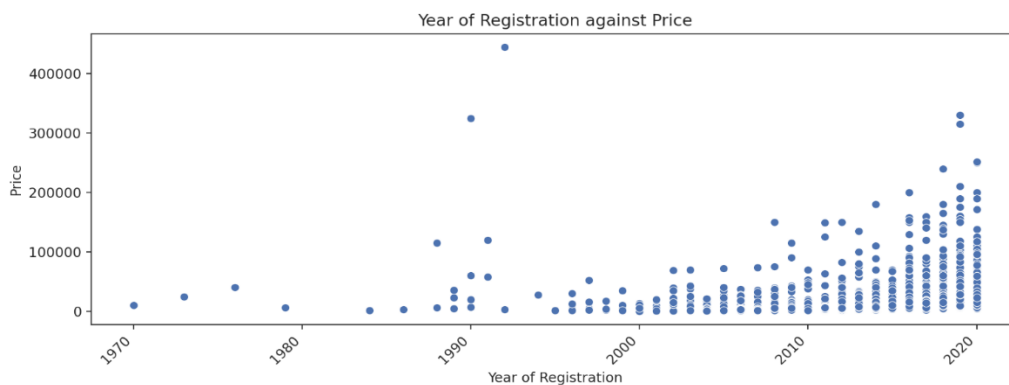


1.2. Analysis of Predictive Power of Features (2-3)

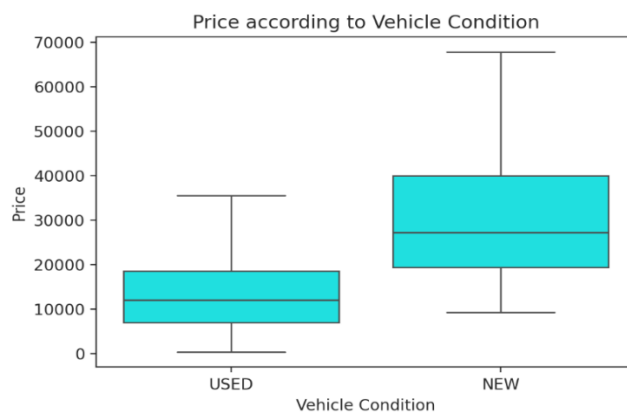
- Mileage vs price sample (20000). There seems to be a linear correlation between mileage and price, the greater the mileage the lower the price.



- Year of registration vs price. Here we see a trend that clearly suggests the newer the vehicle, the higher the price. We also note several exceptions to this rule.



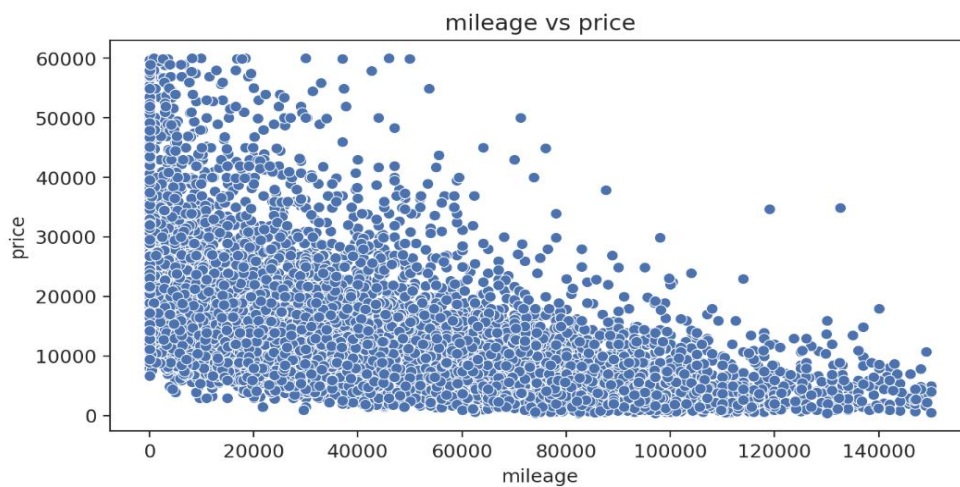
- Vehicle Condition vs price



This boxplot confirms that as expected new vehicles have a higher price in general than used and will be a useful feature for our model.

1.3. Data Processing for Data Exploration and Visualisation (2-3)

1. Capping the price and mileage makes it easier to see the relationship between them.

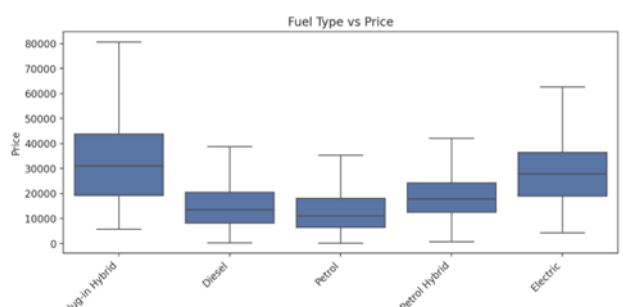
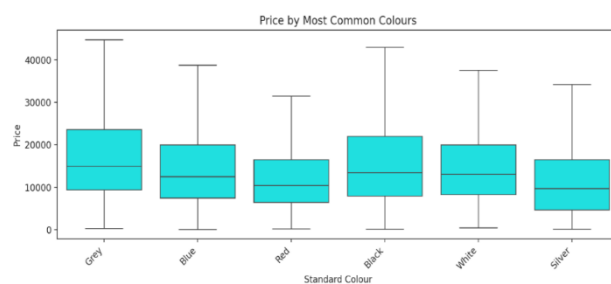


2. Fuel type as a percentage. By normalising the data we see that petrol and diesel vehicles make up over 90% of the data. We also establish that 7/17 body types make up over 99% of the data with Hatchback and SUV dominating.

proportion	
fuel_type	
Petrol	54.04
Diesel	39.39
Petrol Hybrid	3.39
Petrol Plug-in Hybrid	1.53
Electric	1.19
Diesel Hybrid	0.35

```
#Examine body_type distribution
sub_body=adv['body_type'].value_counts
sub_body
```

count	
body_type	
Hatchback	167315
SUV	115872
Saloon	36641



3. We look at the most frequently occurring values. By creating boxplots we conclude the most frequently occurring vehicle colours on average do not impact greatly on the price in comparison with the fuel type or most frequently occurring standard makes.

2. Data Processing for Machine Learning (2 pages, 20%)

2.1. Dealing with Missing Values, Outliers, and Noise (2-3)

1. Year of registration outliers: Vehicle registration began in 1903 so values before this year are incorrect. We use the reg_code column to replace any year of registration values before 1903. We also drop the small percentage of years before 1990 to aid the model.

```
#Replace outlier values with year according to reg_code.
adv.loc[(adv['reg_code'] <= max_year-2000)
        & (adv['year_of_registration'] <= 1903), 'year_of_registration'] = 2000 + adv['reg_code']

[ ] adv.loc[((adv['reg_code'] > 50) &
             (adv['reg_code'] < 50 + (max_year-2000))) &
          (adv['year_of_registration'] <= 1903), 'year_of_registration'] = 2001 - 50 + adv['reg_code']
```

2. Year of registration null values: New vehicles with mileage under 200 can be given the year 2020, which we class as the most recent year. The remaining 584 null values will be dropped rather than risk creating incorrect data. We can now drop reg_code column.

```
[ ] #If vehicle condition is new and mileage <200 replace year of reg values with the most recent year.

adv.loc[(adv['vehicle_condition'] == 'NEW') &
        (adv['year_of_registration'].isnull()) &
        (adv['mileage'] < 200), 'year_of_registration'] = max_year
```

3. Mileage outliers. From the boxplot with the outliers removed we saw the upper bound as roughly 125000 miles. As 99% of the mileage occurs within this limit we cap the mileage.

```
upper_bound = Q3 + 1 * IQR
adv.loc[adv['mileage'] > upper_bound, 'mileage'] = upper_bound
```

We replace the colour null values with the mode colour as colour has little impact on price (we could alternatively drop entire colour column). We drop the null values from the mileage, body_type and fuel_type columns rather than risk creating incorrect data. We still have over 99% of the data remaining.

```
<class 'pandas.core.frame.DataFrame'>
Index: 399454 entries, 0 to 402004
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mileage                               399454 non-null  float64
1   standard_colour                       399454 non-null  object
2   standard_make                         399454 non-null  object
3   standard_model                       399454 non-null  object
4   vehicle_condition                    399454 non-null  object
5   year_of_registration                 399454 non-null  float64
6   price                                399454 non-null  int64
7   body_type                            399454 non-null  object
8   crossover_car_and_van                399454 non-null  bool
9   fuel_type                            399454 non-null  object
dtypes: bool(1), float64(2), int64(1), object(6)
memory usage: 30.9+ MB
```

2.2. Feature Engineering, Data Transformations, Feature Selection (2-3)

1 We change the vehicle condition values from object data to integers and change car and van crossover from Boolean data to integer so we can include them in our model.

```
adv['vehicle_condition'] = adv['vehicle_condition'].replace(['NEW', 'USED'], [0, 1])
adv['crossover_car_and_van'] = adv['crossover_car_and_van'].astype(int)
adv.info()
```

2. One hot coding with max categories 3 for fuel type and standard colour. This converts categorical data to 1's and 0's so we can include them in our model. We know most of the fuel types are either petrol or diesel so 3 categories is enough.

```
ohe = OneHotEncoder(sparse_output=False, max_categories=3)
ohe_1 = ohe.fit_transform(adv[['fuel_type', 'standard_colour']])
ohe_1
```

3. One hot coding with minimum frequency of 1% for body type, standard model and standard make. Due to the high cardinality of categories such as standard_model we set a limit that any type, make or model that represents at least 1% of the data. We add these extra columns and see we now have 63 columns.

```
adv_2 = pd.concat([adv_1.drop(['body_type', 'standard_make', 'standard_model'], axis=1),
                  pd.DataFrame(ohe_2.fit_transform(adv_1[['body_type', 'standard_make', 'standard_model']]))], axis=1)
adv_2.shape

(399454, 63)
```

3. Model Building (2 pages, 20%)

3.1. Algorithm Selection, Model Instantiation and Configuration (3)

A data sample of 50,000 was used for all three of our models.

1. KNN regressor - suitable for linear regression. Using a min max scaler to prevent larger values from dominating. We train the model with 75% of the data and use 25% to test.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=0,  
)  
X_train.shape, y_train.shape, X_test.shape, y_test.shape  
  
((37500, 62), (37500,), (12500, 62), (12500,))
```

The model scores well on the training data with nearly 90% accuracy and slightly lower on the testing data at just over 80% for N=4 nearest neighbours.

```
# interpretation of the score: amount of variance of the target explained by the model  
regr.score(X_train_scaled, y_train), regr.score(X_test_scaled, y_test)  
  
(0.8908994929378892, 0.8237498715318561)
```

2. Using the Decision Tree Regressor model we find a depth of 21 worked well to achieve almost 95% accuracy on the training data and 77% on the testing data with a mean absolute error of 2924.5 (1pd)

```
dtr.score(X_train, y_train), dtr.score(X_test, y_test)  
  
(0.9470833783832997, 0.7727758351120255)
```

```
mean_absolute_error(y_test, dtr.predict(X_test))  
  
2924.5424412692964
```

This discrepancy between testing and training may be due to overfitting.

3. Linear Regression: This model performed least well with the highest mean absolute error. However, there was little difference between the training and testing outcomes, so our model performed well in that respect.

```
mean_absolute_error(y_test, lr.predict(X_test))  
  
4013.682714718916
```

```
lr.score(X_train, y_train), lr.score(X_test, y_test)  
  
(0.7271393840233132, 0.7282561721760097)
```

3.2. Grid Search, and Model Ranking and Selection (3)

1. Hyper-parameter search using grid search to examine different values of N nearest neighbours for our KNN regressor model and see if our initial findings are confirmed.

```
#Grid Search for KNN regressor
# Useful to see the underfitting and overfitting
param_grid = {
    'n_neighbors': [3, 4, 5, 7, 11, 13, 15]
}
```

2. We model the grid search results and rank them. We find that N=5 nearest neighbours gives the best results. High values of N do not improve the model.

	param_n_neighbors	mean_test_score	mean_train_score	std_test_score	std_train_score	rank_test_score
2	5	0.801661	0.871855	0.007272	0.000302	1
1	4	0.799284	0.882766	0.007976	0.000914	2
3	7	0.799161	0.854626	0.007302	0.000812	3
0	3	0.796232	0.899473	0.007828	0.000609	4
4	11	0.784994	0.826344	0.006258	0.000706	5
5	13	0.777283	0.814432	0.006075	0.000795	6

3. We choose KNN regressor with N=5 as our best model.

This is our overall best model where the testing score was highest and the mean absolute error of the testing data the lowest. The

```
mean_absolute_error(y_test, regr.predict(X_test_scaled))
```

2887.3947066666665

Decision Tree regressor was a close second. Further analysis and cross validation of the decision tree regressor suggests the model is overfitting much more than the KNN regressor.

	fit_time	score_time	test_score	train_score
0	0.730070	0.015302	0.738828	0.988456
1	0.635028	0.011420	0.723636	0.989160
2	0.766084	0.007427	0.736901	0.988977

4. Model Evaluation and Analysis (3 pages,)

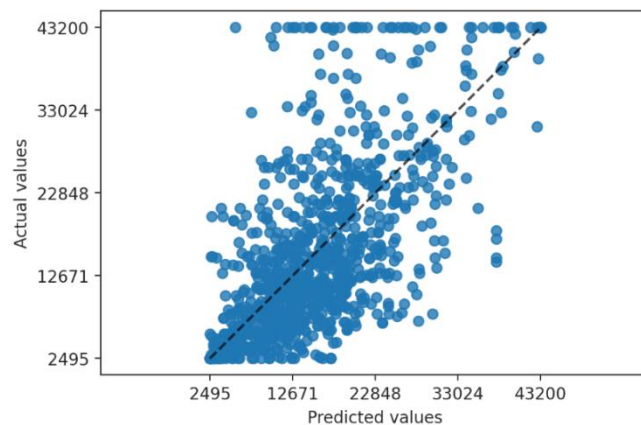
4.1. Coarse-Grained Evaluation/Analysis (3)

1. Cross validation performed on KNN regressor with N=5 shows our model performs well with high mean training and test scores and low standard deviations.

```
scores['train_score'].mean(), scores['train_score'].std()  
(0.8718549810623856, 0.00033815557062391724)
```

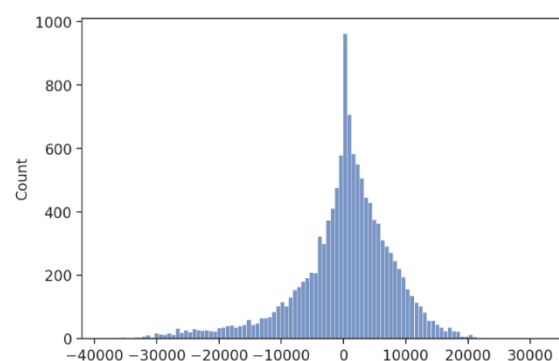
```
| scores['test_score'].mean(), scores['test_score'].std()  
(0.8016611082481946, 0.008130569924273754)
```

2. We plot a graph of predicted values against actual values, we see a general linear correlation that seems to deteriorate as the values increase.



3. If we look at the model residuals we see that as the difference between actual and predicted gets greater the frequency reduces. However we can also see that the model is under/over estimating considerably in some cases.

```
sns.histplot(residuals_test);
```



4.2. Feature Importance (2-4)

1. The model improved significantly when we winsorized the price column

I found that winsorizing the price range with 5% limits greatly improved the KNN regressor model performance. This helped to remove outlier extreme values.

```
from scipy.stats.mstats import winsorize
adv['price'] = winsorize(adv['price'], limits=[0.05, 0.05])
adv['price'].describe().round(2)
```

2. Removing year of registration before 1990 also improved the model. This is probably due to the high numbers of noisy and outlier data. We could try removing all data before the year 2000 to improve the model further.

3. Various one hot coding parameters were also tested before settling on the ones in my report. This could be looked at again to see if we can reduce cardinality and improve the model.

4. Suggestions would be to drop the car and van crossover column since 99% of the data was not a car and van crossover.

proportion	
crossover_car_and_van	
0	99.557897
1	0.442103

4.3. Fine-Grained Evaluation (2-4) (e.g., with instance-level errors)

1. By examining the KNN regressor residuals we see some large differences. Let's examine an instance where the model has under estimated the price by over 40,000.

y_actual	y_pred	residual
43200	2845.8	-40354.2
43200	4696.6	-38503.4
43200	4715.0	-38485.0
43200	4796.8	-38403.2

This vehicle is a Porsche 911 convertible from 2011. In

2011 the mean vehicle price is just under 8,000 and

mean mileage is around 65,000. This shows that the vehicle has a much higher price than average, probably due to the make and a lower than average mileage.

We see the make is Porsche were included in our model columns as they have over 1% frequency, as were convertibles. The

average price for a Porsche in our data is around 35,000 and average year 2012, so it is surprising the model priced it so inaccurately.

Perhaps not enough weight was given to the make and model and body type or possibly our model sample of 50,000 was not reflective enough of the entire data set.

```
adv.loc[[57434, 352430]].T
```

	57434	352430
mileage	45000.0	1852.0
standard_colour	White	Grey
standard_make	Porsche	Peugeot
standard_model	911	2008
vehicle_condition	1	1
year_of_registration	2011.0	2017.0
price	43200	9991
body_type	Convertible	SUV
crossover_car_and_van	0	0
fuel_type	Petrol	Petrol

2. At the other end of the scale we have a vehicle, a 2017 Peugeot SUV with 1852 miles, over priced by around 30,000 by the model. If we look at Peugeot's in general, we see the mean price is 10,000 and mean mileage is around 36,000 and for Peugeot SUV's the mean price is higher at around 15,000. Therefore this SUV vehicle has a low price considering it's low mileage. If we look at vehicles for 2017 as a whole we see the average price is just over 15,000 and if we just look at SUV's the average price is around 20,000 and average mileage nearly 30,000. Considering this is such a low mileage SUV, the model's price prediction is becoming clearer.

This report was created with the use of Google Colab and it's AI generative features.